

01 / START HERE

A creator workflow for Claude

Install the Claude edition from the shared repository. Use 24 workflows and 11 local helpers to turn source material into useful drafts and analysis.

1 / Install in Claude Code

Use a Claude Code client with plugin support. The plugin code is MIT-licensed; your Claude access, usage limits and available tools still apply.

```
/plugin marketplace add ahmedsakri/youtube-agent-pro  
/plugin install youtube-agent-pro
```

Plugin skills use namespaced commands: **/youtube-agent-pro:yt** and **/youtube-agent-pro:yt-script**. The shorter **/yt** form is for skills copied manually into Claude's skill directory.

2 / Add your creator voice, if useful

Your supplied context comes first. Optionally run **/youtube-agent-pro:yt-voice** with your own transcripts or a short interview. The default local profile is **~/claude/youtube/voice.md**. A missing profile does not block scripts, analytics or other work that can proceed from your request.

3 / Ask for a concrete deliverable

```
/youtube-agent-pro:yt-script Write a 45-second English Short  
from this outline. Give me three opening-line options.
```

What you receive

Scripts, plans, copy and analysis. The pack itself does not upload, schedule, edit footage or render video. Review its output before you use it.

02 / CHOOSE THE WORKFLOW

The 24-command map

These are the manual-install command names. For the plugin, add its namespace: /yt becomes /youtube-agent-pro:yt. Start with the hub when the next step is unclear.

Start and find direction

/yt | /yt-voice | /yt-idea | /yt-trend

Route a request, define creator voice, evaluate ideas and plan a calendar.

Write and prepare production

/yt-script | /yt-brief | /yt-edit | /yt-chapters

Hooks, spoken scripts, shot lists, transcript edit decisions and chapters.

Package the video

/yt-package | /yt-thumbnail | /yt-seo

Title/thumbnail pairings, visual concepts, descriptions and search intent.

Read performance

/yt-analytics | /yt-retention | /yt-audit | /yt-funnel

Interpret exports and retention, prioritize fixes and review channel conversion.

Plan and reuse

/yt-shorts | /yt-repurpose | /yt-series | /yt-plan | /yt-endscreen

Clip plans, platform drafts, series structure, capacity and next-video routing.

Research and business

/yt-viral | /yt-collab | /yt-sponsor | /yt-comment

Outlier comparisons, outreach drafts, sponsorship scenarios and comment replies.

Metadata should follow the content

Use yt-seo for a natural title and description that match the video and viewer's likely search. Tags have a limited role; include relevant spelling variants when useful. Hashtags are optional. No keyword list guarantees distribution or trending status.

03 / PUT IT TO WORK

Work from your evidence

Give the workflow your audience, format, language and constraints. A timestamped transcript is more useful than a video URL alone. These examples use the plugin commands.

1 / Extract Shorts from an existing video

```
/youtube-agent-pro:yt-shorts Find three 30-45 second clips.  
Preserve transcript timecodes. Label any new opening line  
as a suggested recording, not words already in the video.
```

2 / Write relevant search metadata

```
/youtube-agent-pro:yt-seo Draft a title and description for  
this phone-camera test. Use accurate search phrases naturally.  
Put the main topic early. Add only relevant tags; no hashtags  
are needed. Do not claim a result the script does not show.
```

3 / Diagnose with data

```
/youtube-agent-pro:yt-analytics Compare these exports with  
similar videos at the same age. Separate Shorts-feed metrics  
from long-form CTR. Give one experiment and the uncertainty.
```

4 / Run a helper yourself

From the cloned repository root, with Python 3 available:

```
python3 skills/yt-script/hookscore.py --hook "I tried three cameras" --json
```

All eleven helpers use the Python standard library and make no network requests. Scores are editing heuristics, not measurements of future views.

04 / USE WITH CONFIDENCE

Setup checks and honest limits

Keep the source files, protect your channel data and separate a useful recommendation from a verified result.

1 / Manual installation alternative

If plugin installation is unavailable, clone the shared repository and copy its root **skills/yt*** folders into `~/.claude/skills/`, or into a project's `.claude/skills/`. Keep all sibling folders together. Check existing names before copying; do not overwrite your own skills blindly.

2 / Troubleshoot the common failures

The command is missing

Reopen Claude Code and inspect the installed plugin. Use `/youtube-agent-pro:yt` for the plugin or `/yt` for a manually copied skill.

The writing does not sound like you

Supply audience, language and style examples in your request, or ask `yt-voice` to create an optional profile at `~/.claude/youtube/voice.md`. You can continue without that file.

A helper cannot find its inputs

Run it from its skill directory, or pass its full path and an absolute input-file path. Keep `hooks.json` and the sibling skill folders in place.

Using a plain Claude chat instead

Paste the relevant `SKILL.md` plus your source material and voice context. A plain chat does not automatically gain local file access or Python execution. Ask for qualitative analysis if the helper cannot run.

The boundaries

Thumbnail output is a concept until a separate image tool creates an asset. Edit decisions need checking against footage. Sponsorship bands are assumptions, not quotes. Public research needs verifiable sources. Publishing needs your request and a separate authorized tool; the pack has no built-in account connection.

Source and updates: github.com/ahmedsakri/youtube-agent-pro

Read the root README and the selected skills for detailed behavior. MIT license notices must remain with redistributed code.